Week 1 (starter): Tokenization Analysis

This is the starter notebook for the Week 1 assignment. It runs as-is on placeholder text so you can see the shape of each step; your job is to replace the placeholders with your own passages and analysis, then commit it to your repository and open a pull request.

Cells marked **TODO (you)** are where you do the work. Everything runs in Jupyter or Google Colab. No GPU, no API key, one dependency: `tiktoken`.

The five parts match the assignment: stand up your repo, run the analysis, evaluate with real figures, find one failure, and submit.

```
# Setup. In Colab, uncomment the install line on first run.
# !pip install tiktoken
import os, pathlib
os.environ['TIKTOKEN_CACHE_DIR'] = str((pathlib.Path('.') / '.tiktoken_cache').resolve())
os.makedirs(os.environ['TIKTOKEN_CACHE_DIR'], exist_ok=True)

import tiktoken
gpt4  = tiktoken.get_encoding('cl100k_base')   # GPT-4 / GPT-3.5
gpt4o = tiktoken.get_encoding('o200k_base')    # GPT-4o
print('tiktoken', tiktoken.__version__, '- encoders ready (cl100k_base, o200k_base)')
```

```
tiktoken 0.14.0 - encoders ready (cl100k_base, o200k_base)
```

Part 1: Stand up your repository

Do this once, outside the notebook:

1. Create a public repo (suggested name `cosc-650`).
2. Add a `README.md` a stranger could read (what it is, how it is organized, the tools you use).
3. Add an agent context file that your AI tool reads, with project context and conventions. `AGENTS.md` is the cross-tool convention; `CLAUDE.md` and `GEMINI.md` are tool-specific variants. Use whichever your tool reads.
4. Work on a branch and open a pull request into `main`. You will do this every week.

Then commit this notebook into the repo and keep going.

Helpers (provided)

Two small functions: count tokens for a string, and show the exact sub-token pieces a word breaks into. The demo uses a line you may recognize.

```
def count_tokens(text, enc):
    return len(enc.encode(text))

def show_split(word, enc=gpt4):
    ids = enc.encode(word)
    pieces = [enc.decode([i]) for i in ids]
    print(f'{word!r:18s} -> {len(ids)} token(s): {pieces}')

# demo: some short strings are a single token; capitalized or rarer words fragment
for w in ['Panic', ' towel', '42', 'antidisestablishmentarianism']:
    show_split(w)
```

```
'Panic'            -> 2 token(s): ['P', 'anic']
' towel'           -> 1 token(s): [' towel']
'42'               -> 1 token(s): ['42']
'antidisestablishmentarianism' -> 6 token(s): ['ant', 'idis', 'establish', 'ment', 'arian', 'ism']
```

Part 2: Your passages

TODO (you): replace the two placeholders with your own text. The non-English passage must be at least 100 words, with a faithful English translation. The placeholders below are short Hitchhiker's Guide lines so the notebook runs; swap in your real passages.

```
# TODO (you): replace both with your own >=100-word passage and its translation.
english_text = """
```

```
Álvarez has neither let Atlético de Madrid down nor disrespected the club, whereas the club and the fans cannot say
Miguel Ángel Gil Marín, the *Colchoneros'* CEO, stated a few days ago that "Julián has been poorly advised since the
As for the *Rojiblanco* fanbase, there is nothing more to add: insults, boos, jeers, scorn, and social media attacks
"""
foreign_text = """
Álvarez no le ha fallado ni le ha faltado al respeto al Atlético de Madrid, mientras que el club y la afición no pue
Miguel Ángel Gil Marín, consejero delegado de los colchoneros, dijo hace algunos días que “Julián ha estado mal ases
Y de la parcialidad rojiblanca ni qué agregar: insultos, abucheos, cánticos, repudio y ataques en redes contra el qu
"""

print('English words:', len(english_text.split()))
print('Foreign words:', len(foreign_text.split()))

def report(label, text):
    print(f'{label:9s} | chars {len(text):4d} | GPT-4 {count_tokens(text, gpt4):4d} | GPT-4o {count_tokens(text, gpt

report('English', english_text)
report('Foreign', foreign_text)

tax_gpt4  = count_tokens(foreign_text, gpt4)  / count_tokens(english_text, gpt4)
tax_gpt4o = count_tokens(foreign_text, gpt4o) / count_tokens(english_text, gpt4o)
print(f'\nMultilingual tax  GPT-4: {tax_gpt4:.2f}x   GPT-4o: {tax_gpt4o:.2f}x')
# TODO (you): one or two sentences interpreting these numbers for YOUR language pair.
```

```
English words: 120
Foreign words: 122
English   | chars  694 | GPT-4  173 | GPT-4o  167
Foreign   | chars  691 | GPT-4  196 | GPT-4o  170

Multilingual tax  GPT-4: 1.13x   GPT-4o: 1.02x
```

Interput Language Pairing

In GPT-4, English uses fewer tokens (173) than Spanish (198), with a multilingual tax of 1.14x. Interestingly, The GPT4o improves the multilingual tax to 1.03x and uses signficanlty less tokens (172) for the Spanish text, while the Englihs is only a slight improvement.

Part 3: Evaluate with real figures

Turn the counts into engineering consequences. The skeleton below computes both; keep it pointed at your real passages.

```
CTX = 128_000
en = count_tokens(english_text, gpt4)
fo = count_tokens(foreign_text, gpt4)
print(f'A {CTX:,}-token window holds about {CTX//en:,} English copies and {CTX//fo:,} foreign copies of your passage
print(f'Per-request cost multiplier for the foreign language: {fo/en:.2f}x (billing is per token).')
# TODO (you): state what this means for a product serving users in your chosen language.
```

```
A 128,000-token window holds about 739 English copies and 653 foreign copies of your passage.
Per-request cost multiplier for the foreign language: 1.13x (billing is per token).
```

What does this mean for a product serving Spanish-language users?

Spanish text with the GPT-4 tokenizer uses more tokens than English, causing the billing to accumulate faster for Spanish-speaking users.

Part 4: Bias splits and one failure

TODO (you): (a) pick three words where your non-English form fragments far worse than the English equivalent, and show both with `show_split`; (b) find ONE input whose token count defies intuition and explain it. A few failure candidates are demonstrated below to get you started; replace them with your own find and write the explanation plus a mitigation.

```
# (a) TODO (you): three real bias pairs from your languages.
print('Spanish:')
for w in ['faltado al respeto', 'temporada', 'abucheos']:
    show_split(w)

print('\nEnglish baseline:')
for w in ['disrespect', 'season', 'boos']:
    show_split(w)
```

```
print('\n(b) failure candidates to explore (replace with your own find):')
# show_split('🚀')                # a rocket emoji
# show_split('hello')                     # baseline
# show_split(' hello')                    # a leading space changes the tokenization
# show_split('https://www.example.com')     # URLs fragment
show_split('restaurant')                            #correct spelling
show_split('resturant')                            #typo
# TODO (you): explain WHY your chosen case behaves this way, and how you would budget or normalize around it.
```

```
Spanish:
'faltado al respeto' -> 7 token(s): ['f', 'alt', 'ado', ' al', ' res', 'pet', 'o']
'temporada'        -> 3 token(s): ['temp', 'or', 'ada']
'abucheos'         -> 3 token(s): ['ab', 'uche', 'os']

English baseline:
'disrespect'       -> 2 token(s): ['dis', 'respect']
'season'           -> 1 token(s): ['season']
'boos'             -> 2 token(s): ['bo', 'os']

(b) failure candidates to explore (replace with your own find):
'restaurant'       -> 1 token(s): ['restaurant']
'resturant'        -> 3 token(s): ['rest', 'ur', 'ant']
```

Why do the bias splits behave this way?

The main reason BPE tokenization behaves this way is that its vocabulary was built from patterns in text, with English receiving more representation than many other languages. Spanish has different grammar and word patterns, so some Spanish words and phrases are less likely to match in the tokenizer's vocabulary.

For example, Spanish does not have a direct one-word translation for `disrespect` in this context, so the equivalent phrase `faltado al respeto` requires more text and breaks into more tokens. This shows that differences in how languages express the same meaning can also affect token counts.

The terms `season` and `boos` show a different pattern. The English word `season` is represented as one token, while `temporada` breaks into three pieces. Similarly, `abucheos` breaks into three pieces while `boos` uses two. These splits show that the tokenizer's vocabulary contains more efficient matches for some English words or patterns than their Spanish equivalents.

Why do misspellings add tokens?

Thinking of issues when I write prompts, it is very easy to make typos or simply misspell words. Typing fast can cause typos, but even missing the first **a** in `restaurant` adds two tokens to the word.

This happens because BPE tokenization only knows patterns from the text used to build its vocabulary. Since the correct spelling, `restaurant`, appears frequently, the tokenizer has learned a token for the entire word. The misspelling `resturant` is not common or accurate, so the tokenizer falls back to reading it in smaller pieces it already knows.

One way to budget or normalize for this would be to use spellcheck on the prompt before it is run. Another option could be grouping long prompts to find meaning before they are run.

Part 5: Submit

Before you open the pull request, check:

- The notebook runs top to bottom on **your** passages, not the placeholders.
- The failure case has a cause and a mitigation.
- The PR description has a one-paragraph result summary with your headline numbers.
- You linked one issue in your repo logging this as a research note (title, inputs, what you found).

Rubric: repo quality (15), counts from both tokenizers (20), tax computed (15), three bias splits (20), cost and context figures (15), the failure case (10), PR hygiene (5).